

Deep Reinforcement Learning

Policy Gradients

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- From value-based methods to direct policy optimization
- Policy gradients
 - The objective
 - Evaluation the objective
 - Differentiating the objective
- The REINFORCE algorithm
- Understanding and challenges
- Reducing the variance
 - Baselines
 - Reward to go

- Off-policy policy gradients
- Some implementation details

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q -learning	Q -learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	
11	Advanced algorithms (Part I): From policy gradient to PPO	
12	Advanced algorithms (Part II): From Q -learning to Soft Actor-Critic	
13	Exploration	
	Model-Based Control	
	Advanced Topics	

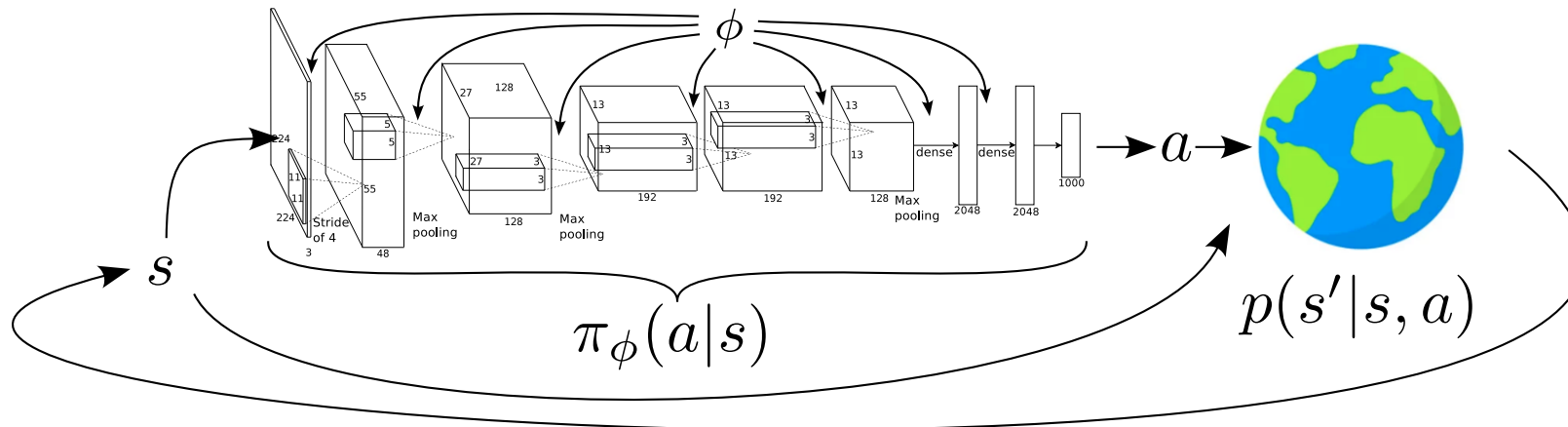
Chapter	Topic	Content
---------	-------	---------

Lecture contents

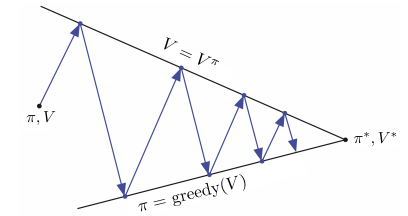
From value-based methods to direct policy optimization

From value-based methods to direct policy optimization

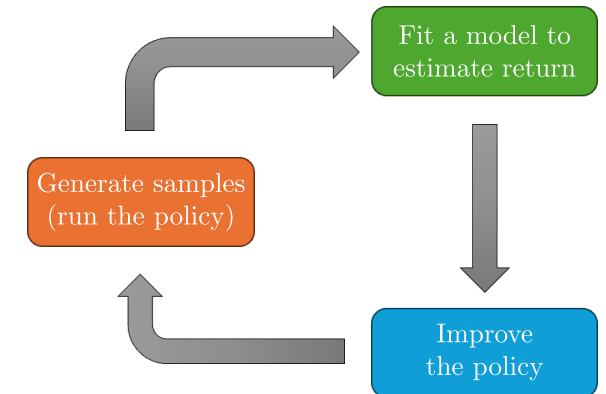
- Until now, all approaches followed the GPI concept.
- These are all **value-based methods**: Need an estimate of V^π or Q^π to improve π .
- But: what are we after in RL, really? $\Rightarrow$ The **optimal policy** π^* !



Inspired by Sergey Levine's CS285 lecture.



GPI (R. S. Sutton and Barto 1998).



Inspired by Sergey Levine's CS285 lecture.

Alternative approach:

1. Define probabilities over entire trajectories.
2. Formulate RL objective; optimize dynamics via policy parameters ϕ .

$$\Rightarrow p_{\phi}(\underbrace{s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T}_{=\tau}) = p_{\phi}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\phi}(a_t | s_t) p(s_{t+1} | s_t, a_t).$$

$$\Rightarrow \phi^* = \arg \max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] = \arg \max_{\phi} \mathbb{E}_{s_0 \sim p(s_0)} [V^{\pi_{\phi}}(s_0)].$$

Policy gradients

The reinforcement learning objective

$$\phi^* = \arg \max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] = \arg \max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)]$$

Infinite horizon case: $\phi^* = \arg \max_{\phi} \mathbb{E}_{(s,a) \sim p_{\phi}(s,a)} [r]$

Finite horizon case: $\phi^* = \arg \max_{\phi} \sum_{t=0}^{T-1} \mathbb{E}_{(s_t, a_t) \sim p_{\phi}(s_t, a_t)} [r_t]$

- Infinite horizon: expected reward according to the stationary distributions of s and a .
- Finite horizon: linearity $\Rightarrow$ expected reward of individual stages (may differ with t).
- We are sticking to the finite-horizon case here.

Evaluating the objective

$$\phi^* = \arg \max_{\phi} \underbrace{\mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right]}_{L_{\pi}(\phi)}$$

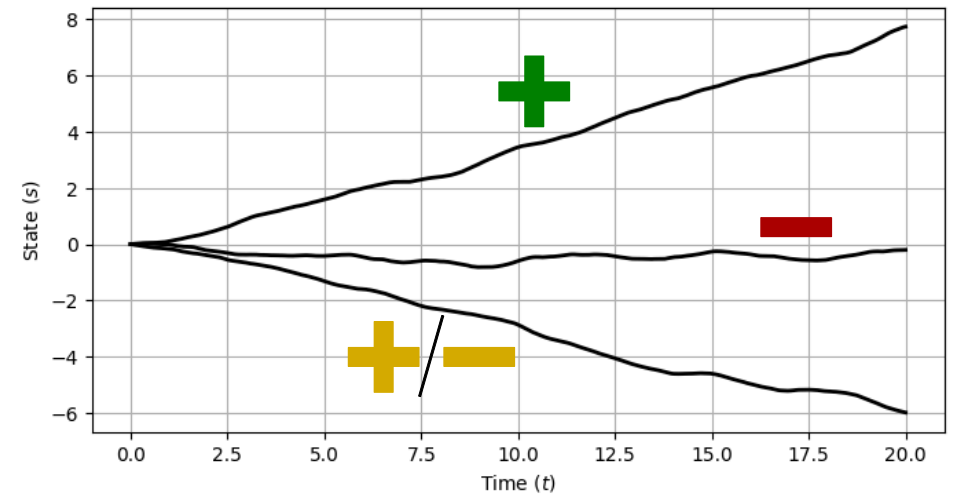
- First, let's think about evaluating $L_{\pi}(\phi)$ for a fixed policy π .
- How do we estimate expectations if we don't have access to a model?

⇒ Monte Carlo sampling!

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} r_{i,t}$$

- Sample N trajectories with s_0 according to the initial distribution and then following π .
- Take the sample average over these trajectories.

Policy gradient: Depending on the return, the plan is to increase or reduce the trajectories' likelihoods by adapting the policy π .



Inspired by Sergey Levine's [CS285 lecture](#).

Differentiating the policy (1)

Definition of the expectation in continuous spaces

The expected value of a function $x(\tau)$ is

$$\mathbb{E}_{\tau \sim p} [x(\tau)] = \int p(\tau) x(\tau) d\tau.$$

Here, p is the density according to which τ is distributed, with $\int p(\tau) d\tau = 1$.

$$\phi^* = \arg \max_{\phi} \underbrace{\mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right]}_{L_{\pi}(\phi)}$$

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\underbrace{r(\tau)}_{=\sum_{t=0}^{T-1} r_t} \right] = \int p_{\phi}(\tau) r(\tau) d\tau$$

$$\begin{aligned} \nabla_{\phi} L_{\pi}(\phi) &= \nabla_{\phi} \left[\int p_{\phi}(\tau) r(\tau) d\tau \right] \\ &= \int \nabla_{\phi} p_{\phi}(\tau) r(\tau) d\tau \end{aligned}$$

Note: Integration and differentiation are *linear* $\Rightarrow$ We can swap!

A convenient identity

$$\nabla_{\phi} p_{\phi}(\tau) = p_{\phi}(\tau) \frac{\nabla_{\phi} p_{\phi}(\tau)}{p_{\phi}(\tau)} = p_{\phi}(\tau) \nabla_{\phi} \log p_{\phi}(\tau) \quad (1)$$

$$\begin{aligned} &= \int p_{\phi}(\tau) \nabla_{\phi} \log p_{\phi}(\tau) r(\tau) \, d\tau \\ &= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau) r(\tau)] \end{aligned} \quad (2)$$

Differentiating the policy (2)

$$\phi^* = \arg \max_{\phi} L_{\pi}(\phi)$$

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)] = \int p_{\phi}(\tau) r(\tau) d\tau$$

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau) r(\tau)]$$

$$\underbrace{p_{\phi}(s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T)}_{=p_{\phi}(\tau)} = p(s_0) \prod_{t=0}^{T-1} \pi_{\phi}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

take log on both sides! $\Downarrow$ $(\log(a \cdot b) = \log(a) + \log(b))$

$$\log p_{\phi}(\tau) = \log p(s_0) + \sum_{t=0}^{T-1} [\log \pi_{\phi}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)]$$

$$\begin{aligned}
\nabla_{\phi} \log p_{\phi}(\tau) &= \nabla_{\phi} \left[\log p(s_0) + \sum_{t=0}^{T-1} [\log \pi_{\phi}(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)] \right] \\
&= \nabla_{\phi} \log p(s_0) + \sum_{t=0}^{T-1} [\nabla_{\phi} \log \pi_{\phi}(a_t|s_t) + \nabla_{\phi} \log p(s_{t+1}|s_t, a_t)] \\
&= \underbrace{\cancel{\nabla_{\phi} \log p(s_0)}}_{=0} + \sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t|s_t) + \underbrace{\cancel{\nabla_{\phi} \log p(s_{t+1}|s_t, a_t)}}_{=0}
\end{aligned}$$

Policy gradient theorem (simplified)

$$\begin{aligned}
\nabla_{\phi} L_{\pi}(\phi) &= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t|s_t) \right) r(\tau) \right] \\
&= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t|s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right]
\end{aligned}$$

Policy gradient theorem

What does this mean?

1. Goal: find a policy that maximizes the value,

$$\max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} \left[\sum_{t=0}^{T-1} r_t \right] = \max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} [r(\tau)].$$

Policy gradient theorem (simplified)

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right]$$

2. Neural network approximator $\pi_{\phi} \Rightarrow$ maximization w.r.t. the policy parameters:

$$\max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)] = \max_{\phi} L_{\pi}(\phi).$$

3. Challenging objective function: Integration over τ (i.e., the space of trajectories) is infeasible!

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)] = \int p_{\phi}(\tau) r(\tau) d\tau.$$

4. Use log identity to derive a formulation of the gradient that we can *approximate using sampling*:

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right].$$

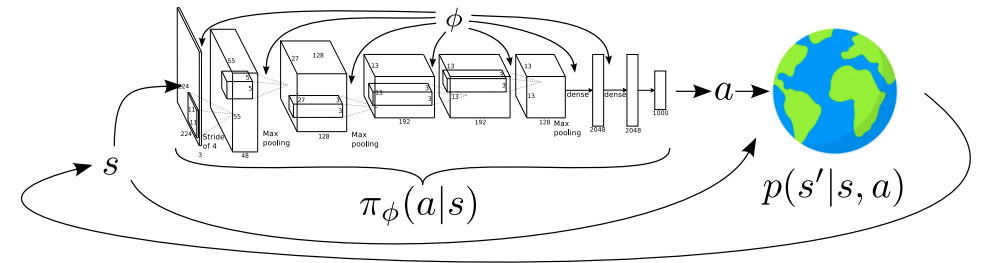
REINFORCE

We now have a formulation of the policy gradient that we can **approximate** using **Monte Carlo sampling**:

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right] \approx \underbrace{\frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \right)}_{(I)} \underbrace{\left(\sum_{t=0}^{T-1} r_{i,t} \right)}_{(II)}.$$

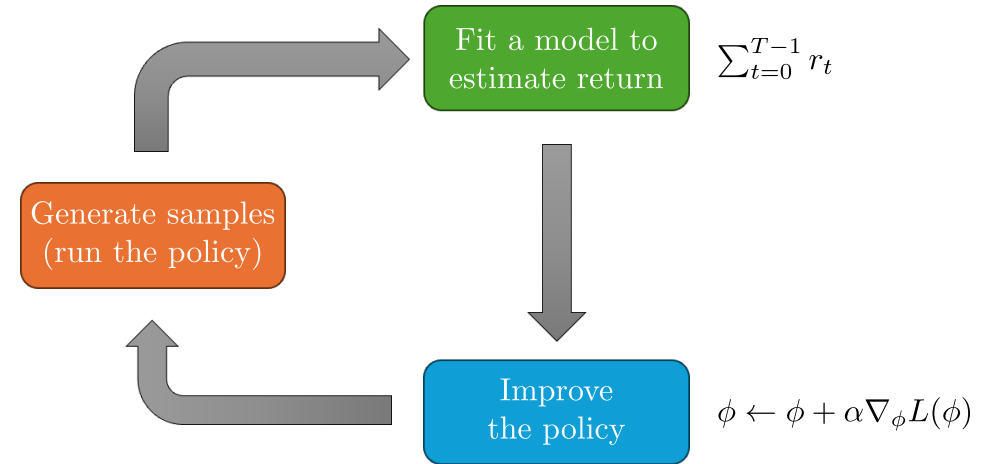
(I) This term is simply the derivative of the policy network w.r.t. the weights $\Rightarrow$ backpropagation!

(II) This term is the experience we collect from interactions with the environment.



The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow \{((s_{i,0}, a_{i,0}, r_{i,0}), \dots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.



Understanding the policy gradient

Continuous action example: A Gaussian policy

- Assume a **Gaussian policy** whose mean is given by a neural network:

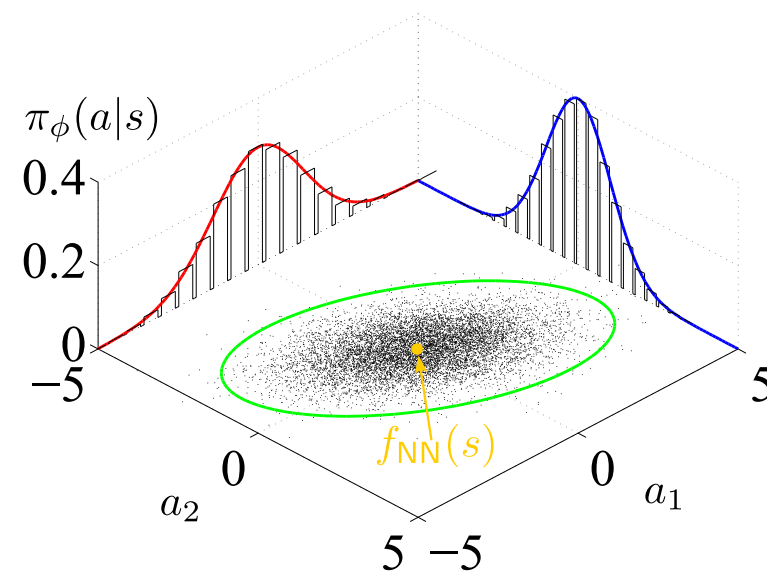
$$\begin{aligned}\pi_{\phi}(a|s) &= \mathcal{N}(f_{\text{NN}}(s), \Sigma) \\ &= \frac{1}{C} \exp \left(-\frac{1}{2} (a - f_{\text{NN}}(s))^{\top} \Sigma^{-1} (a - f_{\text{NN}}(s)) \right) \\ &= \frac{1}{C} \exp \left(-\frac{1}{2} \|a - f_{\text{NN}}(s)\|_{\Sigma}^2 \right).\end{aligned}$$

- Taking the log (with $\log C^{-1} = -\log C$) yields:

$$\log \pi_{\phi}(a|s) = -\frac{1}{2} \|a - f_{\text{NN}}(s)\|_{\Sigma}^2 - \log C.$$

- Taking the gradient leads to the following expression:

$$-\frac{1}{2} \Sigma^{-1} (f_{\text{NN}}(s) - a) \frac{\partial f_{\text{NN}}}{\partial \phi}.$$



Adapted from Wikipedia.

The REINFORCE algorithm

- Sample $\{\tau_i\}_{i=1}^N$ using $\pi_{\phi}(a|s) \Rightarrow \{((s_{i,0}, a_{i,0}, r_{i,0}), \dots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
- $\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right)$.
- Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_{\phi} L_{\pi}(\phi)$.

Comparison to maximum likelihood

- Consider the task of maximizing the likelihood L of a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ in supervised learning, e.g., using a neural network with parameter vector θ :

$$\begin{aligned}\max_{\theta} L(\theta) &= \max_{\theta} p_{\theta}(y_1|x_1) \times \dots \times p_{\theta}(y_N|x_N) \\ &= \max_{\theta} \prod_{i=1}^N p_{\theta}(y_i|x_i).\end{aligned}$$

- Optimizing over products is hard $\Rightarrow$ take the log:

$$\log L(\theta) = \log \left(\prod_{i=1}^N p_{\theta}(y_i|x_i) \right) = \sum_{i=1}^N \log p_{\theta}(y_i|x_i).$$

Maximum likelihood gradient:

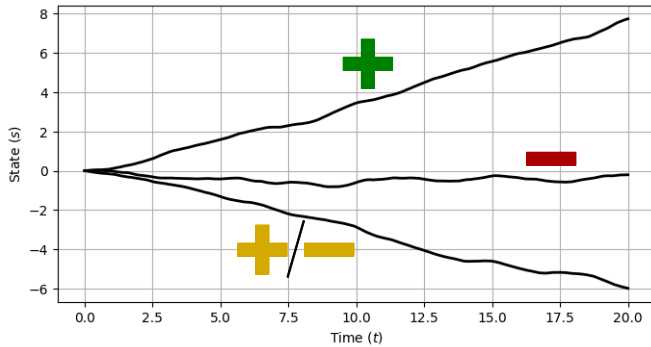
$$\nabla_{\theta} \log L(\theta) = \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(y_i|x_i).$$

Comparison against the policy gradient (MC sampling):

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right).$$

- The policy gradient can be seen as a weighted version of the maximum likelihood objective.
- Why is weighting is necessary?
 - In maximum likelihood, we always want to maximize the likelihood of the “correct” labels.
 - In policy gradients, we may also want to reduce the likelihood of low-reward samples!

Understanding the policy gradient



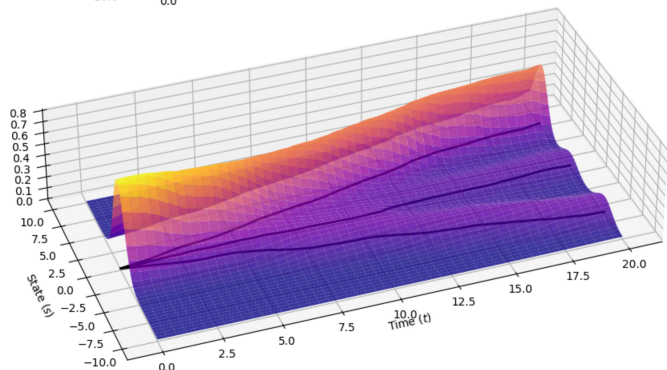
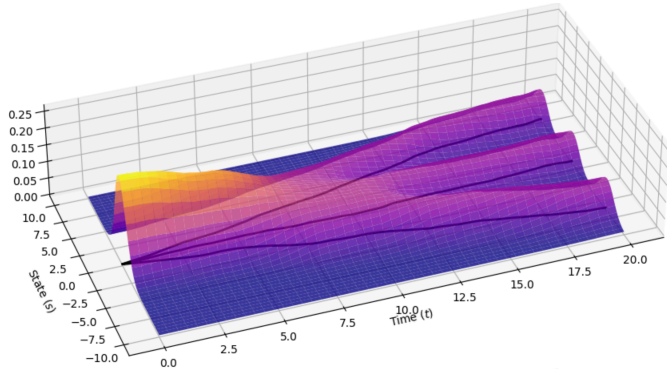
- What did we just do?
⇒ let's rewrite the formula a little and *compare against maximum likelihood*:

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\phi} \log \pi_{\phi}(\tau_i)}_{\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t})} \underbrace{r(\tau_i)}_{\sum_{t=0}^{T-1} r_{i,t}} \quad \text{vs.} \quad \nabla_{\theta} L(\theta) = \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau_i).$$

- Good experience is made more likely: We increase the probability of the policy to produce similar trajectories.
- Bad experience is made less likely.
- This simply formalizes the notion of “trial and error”!

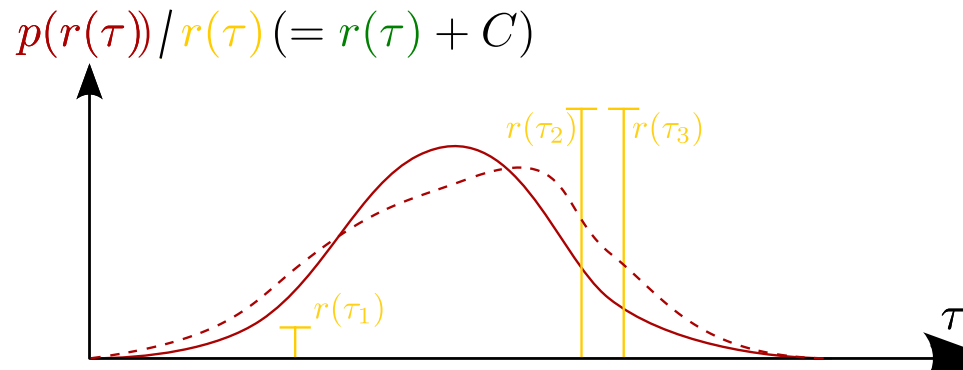
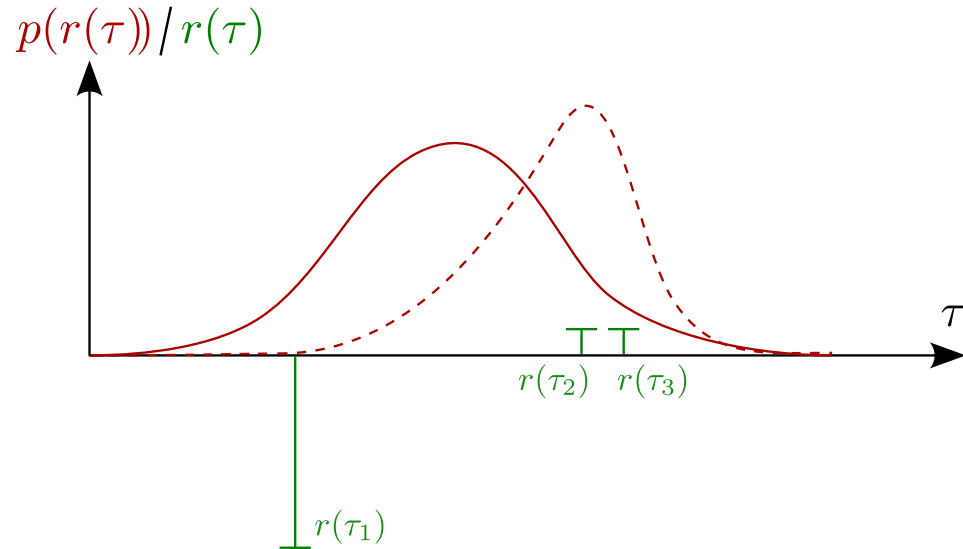
A note on partial observability (without going into details)

The policy gradient also holds for partially observed MDPs (POMDPs). That is, for policies $\pi(a|o)$. In simple terms, the reason is that the policy gradient theorem does not make use of the Markov property.



Challenges with policy gradients

High variance



Inspired by Sergey Levine's CS285 lecture.

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\phi} \log \pi_{\phi}(\tau_i) r(\tau_i).$$

- Adding a constant to the reward should not change the optimal policy!
 - This is true for *any* optimization problem, e.g.,

$$\arg \min_x f(x) = \arg \min_x [f(x) + 1000].$$

- In the limit $N \rightarrow \infty$, this is true for policy gradients as well...
 - ... but it does not hold for finite sample sizes, in particular few sample trajectories.
- Depending on the sign of $r(\tau)$, we **either increase or decrease** the probability of seeing similar trajectories τ_i (and their rewards $r(\tau_i)$) in the future.
- This is an instance of the **high variance** issue with policy gradients.
- An even more “catastrophic” version of this: What if we scale some of the rewards to be exactly zero?

Reducing variance – baselines

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\phi} \log \pi_{\phi}(\tau_i) r(\tau_i).$$

- **Question:** Is there a systematic way to fix the challenge of reward offsets?

⇒ Let's balance the “weight” of our likelihood objective in such a way that ...

- Better-than-average rewards *increase* the probability of the respective trajectories.
- Worse-than-average rewards *decrease* the probability.

- **Several advantages!**

- This aligns with our intuition that better-than-average is reinforced and worse-than-average is penalized.
- This approach makes the policy gradient independent of the actual size of the reward signal.

Approach: subtract a **baseline** b from the reward signal

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\phi} \log \pi_{\phi}(\tau_i) [r(\tau_i) - b],$$

where $b = \frac{1}{N} \sum_{i=1}^N r(\tau_i)$.

Theorem

Subtracting any constant b is *unbiased in expectation*

Proof: Start with the exact formulation of the policy gradient following (2),

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau)(r(\tau) - b)].$$

Unbiased in expectation $\Rightarrow$ baseline term is zero in expectation:

$$\begin{aligned} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau)b] &= \int p_{\phi}(\tau) \nabla_{\phi} \log p_{\phi}(\tau) b \, d\tau \\ &\stackrel{(1)}{=} \int \nabla_{\phi} p_{\phi}(\tau) b \, d\tau = b \nabla_{\phi} \int p_{\phi}(\tau) \, d\tau = (\nabla_{\phi} 1)b = 0. \quad \square \end{aligned}$$

The optimal baseline

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau) (r(\tau) - b)].$$

How do we find the optimal baseline?
 $\Rightarrow$ optimization: minimize the variance

$$\text{var}[x] = \mathbb{E}[x^2] - \mathbb{E}[x]^2$$

$$\text{var} = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\underbrace{(\nabla_{\phi} \log p_{\phi}(\tau) (r(\tau) - b))}_{=g(\tau)}^2 \right] - \left(\mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\nabla_{\phi} \log p_{\phi}(\tau) (r(\tau) - \cancel{b}) \right] \right)^2 \quad (\text{unbiased baseline})$$

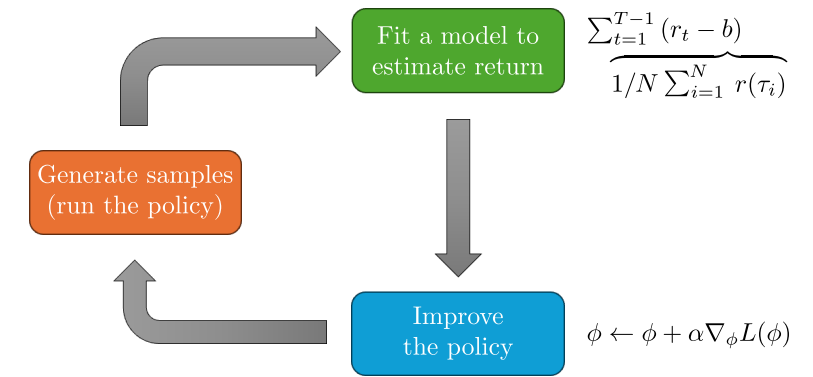
$$\begin{aligned} \frac{d\text{var}}{db} &= \frac{d}{db} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 (r(\tau) - b)^2] = \frac{d}{db} (\mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 r(\tau)^2] - 2\mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 r(\tau) b] + \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 b^2]) \\ &= \frac{d}{db} \left(\cancel{\mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 r(\tau)^2]} - 2b \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 r(\tau)] + b^2 \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2] \right) \\ &= -2\mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2 r(\tau)] + 2b \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [g(\tau)^2] \stackrel{!}{=} 0. \end{aligned}$$

The optimal baseline b^* is the expected reward, weighted by gradient magnitudes:

$$b^* = \frac{\mathbb{E}_{\tau \sim p_\phi(\tau)} [g(\tau)^2 r(\tau)]}{\mathbb{E}_{\tau \sim p_\phi(\tau)} [g(\tau)^2]}.$$

Note: b^* is hard to calculate. In practice: usually take average reward

$$b = \mathbb{E}_{\tau \sim p_\phi(\tau)} [r(\tau)] \approx \frac{1}{N} \sum_{i=1}^N r(\tau_i).$$



Inspired by Sergey Levine's [CS285 lecture](#).

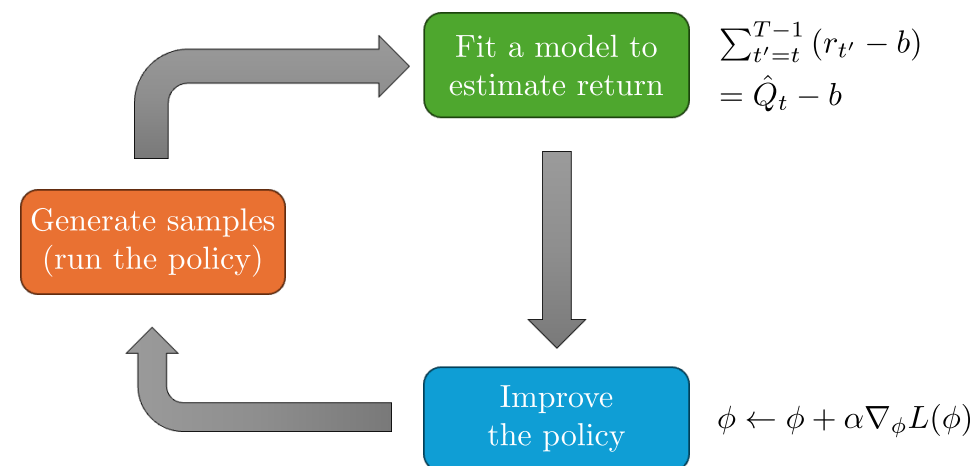
Reducing variance – causality

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

- **Causality:** Policy at time t' cannot affect reward at time t when $t < t'$.
 - “What you do now, is not going to change the rewards you received in the past.”
- **Question:** Are we making use of causality in the above equation?
 - Let's rewrite it and make use of the distributive property ($a \cdot (b + c) = a \cdot b + a \cdot c$):

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

- ⇒ Past rewards (i.e., $t' < t$) have an impact on the policy π_{ϕ} !
- In expectation, these factors have to cancel out (and one can prove this). **But:** for finite sample sizes, they do not and instead increase the variance.



Inspired by Sergey Levine's [CS285 lecture](#).

💡 Do not confuse causality with the Markov property! The Markov property (which may hold for a system, but does not have to) says that your future states do not depend on past states, just the present state. Causality is always true: “Rewards in the past are independent of decisions in the present.”

- **Simple fix:** “reward to go” $\hat{Q}_{i,t} = \sum_{\textcolor{red}{t'}=t}^{T-1} r_{i,t'}$ (the only change is $0 \rightarrow t$),

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \hat{Q}_{i,t}.$$

Off-policy policy gradients

Why policy gradients are on-policy

- Recall the policy gradient optimization problem:

$$\phi^* = \arg \max_{\phi} L_{\pi}(\phi) = \arg \max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)].$$

- The corresponding policy gradient (for simplicity, without baseline) is

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [\nabla_{\phi} \log p_{\phi}(\tau) r(\tau)].$$

- The problem: $\mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)]$ requires on-policy sampling!
- We cannot skip step 1 of the REINFORCE algorithm.

The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_{\phi}(a|s) \Rightarrow \{((s_{i,0}, a_{i,0}, r_{i,0}), \dots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_{\phi} L_{\pi}(\phi)$.

The trouble in Deep RL:

- Neural networks usually require small update steps ...
- ... but a large number of steps!
- On-policy learning can become highly inefficient!

Importance sampling (continuous setting)

Importance sampling

How do we calculate the expectation w.r.t. a different distribution?

$$\begin{aligned}\mathbb{E}_{x \sim p} [f(x)] &= \int p(x) f(x) \, dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) \, dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) \, dx \\ &= \mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} f(x) \right].\end{aligned}$$

💡 This formula is exact!

- Back to the reinforcement learning objective

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} [r(\tau)].$$

- What if we have trajectory samples from $\bar{p}$ instead of p_{ϕ} ?
 - Examples are previous policies, expert demonstrations, combinations thereof, ...
- Simply transfer of the importance sampling formula:

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\phi}(\tau)}{\bar{p}(\tau)} r(\tau) \right].$$

- Insert definition of trajectory probabilities:

$$p_{\phi}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\phi}(a_t | s_t) p(s_{t+1} | s_t, a_t),$$

$$\frac{p_{\phi}(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(s_0)} \prod_{t=0}^{T-1} \pi_{\phi}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}}{\cancel{p(s_0)} \prod_{t=0}^{T-1} \bar{\pi}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}} = \prod_{t=0}^{T-1} \frac{\pi_{\phi}(a_t | s_t)}{\bar{\pi}(a_t | s_t)}. \quad (3)$$

Off-policy policy gradient with importance sampling (1)

- Once more, recall the loss function $L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} [r(\tau)]$.
- **Question:** Can we estimate L for ϕ' using samples according to p_ϕ ?

A convenient identity

$$p_\phi(\tau) \nabla_\phi \log p_\phi(\tau) = \nabla_\phi p_\phi(\tau)$$

$$L_\pi(\phi') = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\frac{p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau) \right]$$

- Note that only $p_{\phi'}(\tau)$ actually depends on the new parameter vector ϕ' :

$$\nabla_{\phi'} L_\pi(\phi') = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\frac{\nabla_{\phi'} p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau) \right] = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\frac{\nabla_{\phi'} p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau) \right] = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\frac{p_{\phi'}(\tau)}{p_\phi(\tau)} \nabla_{\phi'} \log p_{\phi'}(\tau) r(\tau) \right].$$

- It's the policy gradient formula we know, modified by the importance sampling weights!
 - In fact, that's an alternative way to derive it: introduce importance sampling in the policy gradient formula $\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} [\nabla_\phi \log p_\phi(\tau) r(\tau)]$.
- Setting $\theta' = \theta$, we recover the on-policy policy gradient.

💡 As derived earlier, using $p_{\phi'}(\tau)$ or $\pi_{\phi'}(\tau)$ (i.e., the product over action probabilities along the trajectories) yields the same result

$$\nabla_\phi \log p_\phi(\tau) = \cancel{\nabla_\phi \log p(s_0)} + \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t | s_t) + \cancel{\nabla_\phi \log p(s_{t+1} | s_t, a_t)} = \nabla_\phi \log \pi_\phi(\tau).$$

Off-policy policy gradient with importance sampling (2)

- Now that we have the formula, let's reformulate and introduce causality again:

$$\begin{aligned}\nabla_{\phi'} L_{\pi}(\phi') &= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\frac{p_{\phi'}(\tau)}{p_{\phi}(\tau)} \nabla_{\phi'} \log \pi_{\phi'}(\tau) r(\tau) \right] \\ &= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\underbrace{\left(\prod_{t=0}^{T-1} \frac{\pi_{\phi'}(a_t | s_t)}{\pi_{\phi}(a_t | s_t)} \right)}_{\text{Eq. (3)}} \left(\sum_{t=0}^{T-1} \nabla_{\phi'} \log \pi_{\phi'}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right]\end{aligned}$$

(Causality: Future actions don't affect (1) the **current weights** and (2) **past rewards**. $\Rightarrow$ Distribute weights!)

$$= \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi'} \log \pi_{\phi'}(a_t | s_t) \left(\prod_{t'=0}^t \frac{\pi_{\phi'}(a_{t'} | s_{t'})}{\pi_{\phi}(a_{t'} | s_{t'})} \right) \right) \left(\sum_{t=0}^{T-1} r_t \left(\prod_{t''=t}^{t'} \frac{\pi_{\phi'}(a_{t''} | s_{t''})}{\pi_{\phi}(a_{t''} | s_{t''})} \right) \right) \right]$$

- Ignoring the last term yields some form of policy iteration algorithm (more on this later):

$$\left(\prod_{t''=t}^{t'} \frac{\pi_{\phi'}(a_{t''} | s_{t''})}{\pi_{\phi}(a_{t''} | s_{t''})} \right).$$

Some implementation details

Policy gradient with automatic differentiation

$$\nabla_{\phi} L_{\pi}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \underbrace{\hat{Q}_{i,t}}_{=\sum_{t'=t}^{T-1} r_{i,t'}}.$$

- Calculation of $\nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t})$ can be very inefficient!
 - Consider a network with $d = 10^6$ weights and a dataset of $N = 100$ trajectories of length $T = 100$ each.
 - We would have to calculate a vector with 10^6 entries $NT = 10^5$ times!
- **Alternative:** Compute policy gradients using automatic differentiation (e.g., Python's pyTorch, TensorFlow, JAX).
 - We need a graph such that its gradient is the desired policy gradient.
 - Recall the maximum likelihood objective from supervised ML:

$$\nabla_{\phi} J_{\text{ML}}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t} | s_{i,t}), \quad \Rightarrow \quad J_{\text{ML}}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \log \pi_{\phi}(a_{i,t} | s_{i,t}).$$

Pseudocode example

```
# Define a simple policy network (actor)
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 64),
            nn.ReLU(),
            nn.Linear(64, action_dim)
        )

    def forward(self, state):
        logits = self.fc(state)
        return Categorical(logits=logits) # Outputs a policy distribution

# Initialize network and optimizer
policy = PolicyNetwork(state_dim=n, action_dim=m)
optimizer = optim.Adam(policy.parameters(), lr=0.01)

# Forward pass: get the action distribution for the current state
dist = policy(state)

# Calculate the log probability of the action that was actually taken
log_prob = dist.log_prob(action_taken)

# Define the Pseudo-Loss: -log_prob * Reward ("-" due to gradient descent)
pseudo_loss = -(log_prob * reward)

# Autodiff calculates the policy gradient
pseudo_loss.backward()
optimizer.step()
```

- **Solution:** Just implement a *pseudo-loss* as a weighted maximum likelihood

$$\tilde{L}(\phi) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \log \pi_{\phi}(a_{i,t} | s_{i,t}) \hat{Q}_{i,t}.$$

Policy gradient in practice

Some practical considerations / differences to supervised learning

- Remember that the **policy gradient** has high variance.
 - This isn't the same as supervised learning!
 - Gradients will be really noisy!
- Consider using **much larger batches**.
 - Several 1000s is quite common.
- Tweaking learning rates is very hard.
 - Adaptive step size rules like ADAM can be OK.
 - More details later.

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.

Sutton, Richard S, David McAllester, Satinder Singh, and Yishay Mansour. 1999. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." In *Advances in Neural Information Processing Systems*, edited by S. Solla, T. Leen, and K. Müller. Vol. 12. MIT Press.